CSE 3421
SECURITY

Samin Sharaf Somik
Lecturer, Dept of CSE, UIU

Why this lesson?

An unsecured system is not much attractive for the users, but it is for the hackers

Properties of Secure Softwares

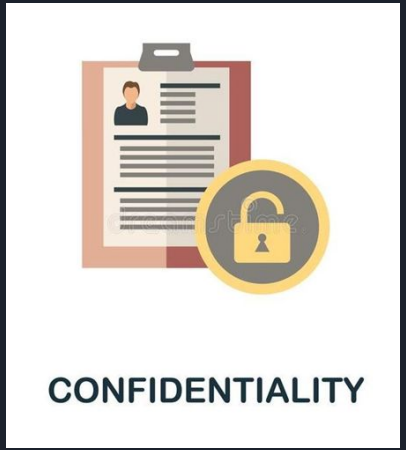
1. Confidentiality
2. Integrity
3. Authentication
4. Non-repudiation
5. Availability



Confidentiality

- The ability of a system to ensure that an asset is **viewed** only by authorized parties
- Sensitive information is not leaked to unauthorized parties
- Privacy for individuals, confidentiality for data

- Examples
  - Exam assignments should not be published (at least until the exam starts)
  - GPA should only be visible to the particular student involved



Integrity

- The ability of a system to ensure that an asset is **modified** only by authorized parties
- Sensitive information is not damaged by unauthorized parties

- Examples
  - Submissions are not edited by anyone other than student
  - Grades are determined only by instructor or auto-grader



Authentication

- The ability to **verify** the identity of an individual or entity.
- Verify the identity of a person (or other external agent) making a request of a computer system



- Example
  - Should be able to determine if the user is indeed a student or an instructor

Nonrepudiation / Accountability

- The ability of a system to confirm that a doer cannot convincingly deny having done something
- Note: opposite of privacy/anonymity; requires a balance

- Example
  - Student cannot deny to have edited submission after the deadline



Availability

- The ability of a system to ensure that an asset can be used **all the time (24/7)** by any authorized parties
- A system is responsive to requests, system does not get blocked out
- Sometimes unauthorized control over the system makes it unavailable, must be available all the time
- Another scenario is DOS (Denial of Service)



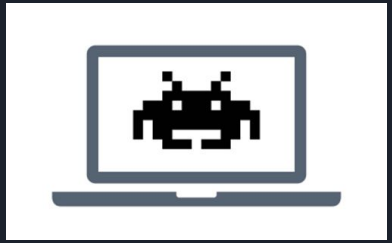
Popular Cyber Attacks

1. Malware
2. Password Attack
3. SQL Injection
4. Zero Day Attack



Malware

- Unwanted software that is installed on your system without your consent.
- **Ransomware**: This is malicious software that holds your data hostage until a ransom is paid
- **Spyware:** These are programs installed to collect confidential information about users
- **Trojan horse:** A seemingly legitimate program, but with a malicious intent
- **Logic Bomb:** This type of virus is capable of being triggered at a precise moment



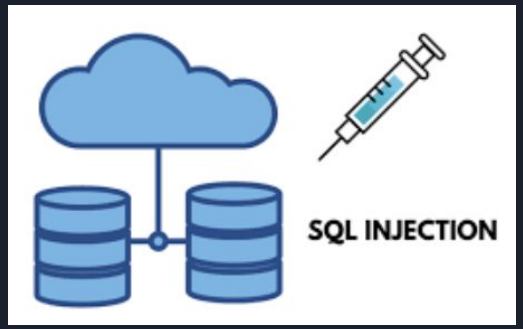
Password Attacks

- **Brute Force / Dictionary Attack:** Try by using common passwords
- **Phishing:** Sending fraudulent but attractive emails
- **Man-in-the-middle:** Sniffing the communication packets
- **Credential Stuffing**: Signed in from another device
- **Keyloggers**: Sniffing while typing the password
- **Social Engineering:** Manipulating into giving up the password



SQL Injection

- A code injection technique that might destroy your database
- One of the most common web hacking techniques
- The placement of malicious code in SQL statements, via web page input
- SQL injection usually occurs when you ask a user for input, like their username/userid, and instead of a name/id, the user gives you an SQL statement that you will unknowingly run on your database



SQL Injection

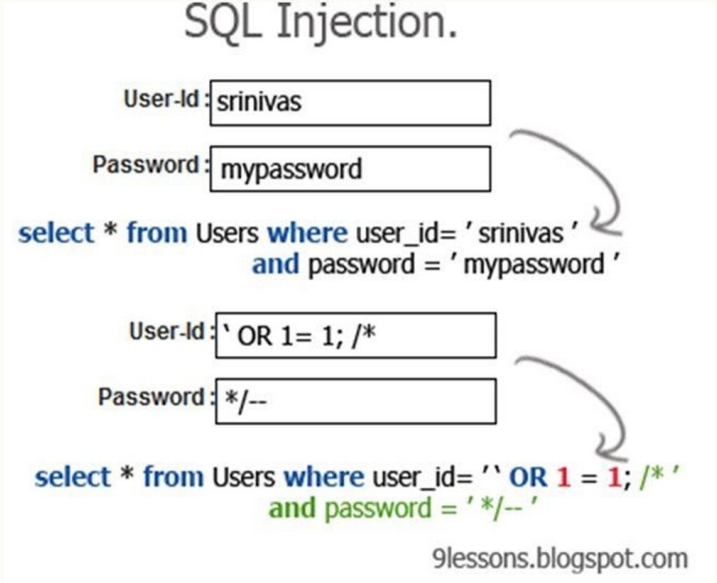
- SQL Injection Based on 1=1 is Always True
  - UserId: 105 OR 1=1
  - SELECT * FROM Users WHERE UserId = 105 OR 1=1;

- SQL Injection Based on ""="" is Always True
  - UserName: " or ""=" | Password: " or ""="
  - SELECT * FROM Users WHERE Name ="" or ""="" AND Pass ="" or ""=""

- The SQL above are valid and will return all rows from the "Users" table

SQL Injection

Example of SQL injection



SQL Injection

- To protect a web site from SQL injection, you can use SQL parameters.
- SQL parameters are values that are added to an SQL query at execution time, in a controlled manner.
- parameters are represented in the SQL statement by a @ marker.
- The SQL engine checks each parameter to ensure that it is correct for its column and are treated literally, and not as part of the SQL to be executed.
- SELECT * FROM Users WHERE UserId = @0

Zero Day Attack

- Zero day means very recent
- Software developers are always looking out for vulnerabilities to "patch" - that is, develop a solution that they release in a new update
- Sometimes hackers spot the vulnerability before the software developers do
- Then they can write and implement a code to take advantage of it
- Also sometimes users ignore the recent software updates. All these cause zero day attack

Excuses from Practitioners

- “It's not exploitable.”
- *“No one will do that!”*
- *“Why would anyone do that?”*
- *“We've never been attacked.”*
- *“We're secure, we use cryptography.”*
- *“We're secure, we use a firewall.”*
- “We've reviewed the code, and there are no security bugs.”

Good Practices

- **Fields length** checking
- **Validate** input not only on client-side but on **server-side** environment too
- Avoid SQL Injection attacks using proper preventive measures
- Possibly use **high level virtualized languages such as Java, C#**
- Low level languages like C and C++ are more exposed to buffer overflow exploits
- Always deploy your application in the authentic Appstore

Good Practices

- Use **Public Key Cryptography (AES, DES)** to do effective encryption
- Facilitate password strength and prompt to change old passwords
- **Identification & Authentication** have to be done over **encrypted channels (e.g., HTTPS)**
- Do security testing

References

- Slides adapted from Md. Rafi-Ur-Rashid, Lecturer, Dept. of CSE, UIU
- SQL Injection: https://www.w3schools.com/sql/sql_injection.aspction
- Zero Day Attack: https://www.kaspersky.com/resource-center/definitions/zero-day-exploit Attacks



Thank You ^_^ !!!